Runspindle: The Case for Verified Work

Design scaffold; no live deployment or measured savings.

November 2026

The problem

- A tool call is not an accepted outcome.
- Human repair, retries, and uncertain external effects belong in the cost.
- A harness must preserve task authority rather than expand it.

The proposed system

- Core: Rust and Temporal. Host: native Codex.
- Browser: OpenBao, PinchTab, KasmVNC. Tether: existing Slack integration.
- Refinery: Python DSPy. Console: actual fleet views. Platform: distribution.
- All new integrations remain scaffolded.

A measurable claim

- Compare native, expert-guided, and harness-assisted work on matched tasks.
- Predeclare acceptance, authorization, failure, and stop rules.
- Count every attempt and human intervention. No results are asserted.

Refinement with gates

- Select and authorize; review and redact; curate grouped held-out data.
- Compare candidates on quality, safety, latency, and full cost.
- Bind attributable reactions/replies to exact outputs; task-specific gold rules quarantine conflicts.
- Gold eligibility is separate from human-approved candidate release and rollback.

Economics without a forecast

- Divide total operating cost by verified authorized completions.
- Include setup, maintenance, verification, recovery, and failed attempts.
- Low volume or provider improvements can erase a proposed advantage.

Next evidence

- Demonstrate a permissioned end-to-end task and uncertain-result recovery.
- Verify component contracts before any production claim.
- Historical source evidence is separate from Runspindle results. Source: SRC-002, SRC-013.